# Deploying 3D Diffusion Policy for Desktop Manipulation

Yitao Zeng Yufeng Wen Chuyao Fu

Southern University of Science and Technology, Shenzhen, China

Course Instructor: Max Q.-H. Meng

**Abstract**

This report documents the implementation and evaluation of a 3D Diffusion Policy pipeline on an ARX robotic arm. The system combines teleoperation demonstrations, depth-camera calibration, point-cloud preprocessing, policy training, and ROS-based execution. A practical focus is converting noisy depth observations into a compact, robot-aligned representation while maintaining consistent data and action interfaces between training and deployment. In the recorded evaluation, 20 physical trials per task produced success rates of 70% for bowl picking, 100% for drawer closing, 30% for placing a lemon in a bowl, and 10% for the lemon–drawer sequence. These results show a substantial gap between short behaviors and multi-step execution. The report describes the implementation choices, preserves the task-level observations, and discusses persistent policy jitter and evaluation limitations. It presents a deployment study using an existing policy framework rather than a new diffusion architecture or a controlled comparison against alternative methods.

**Keywords:** imitation learning; 3D Diffusion Policy; point clouds; robotic manipulation; real-robot deployment.

## 1 Objective and Contribution

A learned manipulation policy must operate through a concrete sensing, data, and control pipeline. A mismatch in camera coordinates, observation preprocessing, or action representation can invalidate an otherwise usable trained model. This project examines that integration problem for desktop manipulation using 3D Diffusion Policy (DP3) [2].

The work consists of building the physical data-to-action pipeline, replacing a slow point-cloud preprocessing attempt with workspace cropping and fixed-size sampling, and evaluating increasingly complex tasks on the robot. DP3 and Diffusion Policy [1] provide the underlying learning formulation; their architectures and general capabilities are not claimed as original contributions of this project. The empirical question is how the implemented system behaves on the recorded desktop tasks, especially when a task requires several sequential actions.

Yitao Zeng's work included point-cloud preprocessing and participation across the integrated teleoperation, calibration, training, and ROS deployment pipeline. The experiments were part of a team project, and the task results describe system-level performance.

## 2 System and Demonstration Pipeline

### 2.1 Hardware and data interfaces

The setup uses an ARX 5 arm with bilateral teleoperation and a RealSense D435-family depth camera (Fig. 1).

Teleoperation provides task demonstrations. The recorded data fields include depth images, end-effector actions, point clouds, and robot joint positions, stored as episodic arrays in Zarr. The source describes six-dimensional pose actions and a seven-element joint-position vector. The exact rotation parameterization and action-frame convention must be recovered from the collection and deployment code rather than inferred from those dimensions alone.

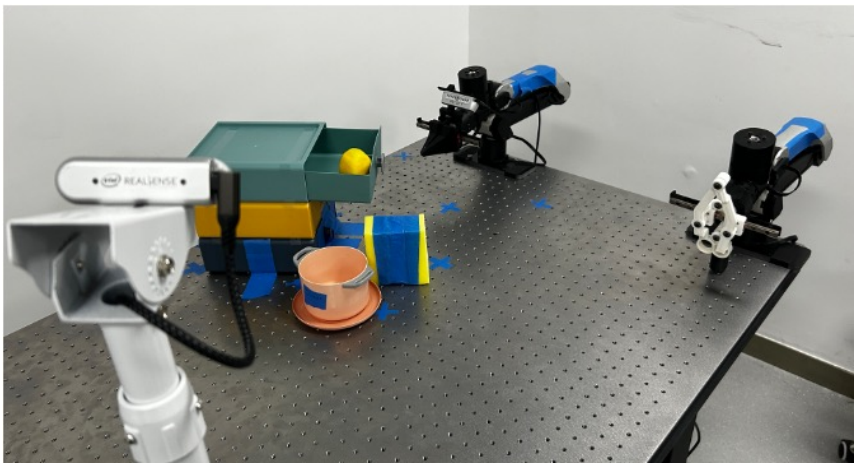


**Figure 1:** Original physical setup for teleoperation data collection and learned-policy execution.

## 2.2 Camera calibration and coordinate consistency

Intrinsic calibration uses checkerboard observations from multiple viewpoints. The calibration procedure uses a checkerboard with 15 mm squares and approximately 15–20 images.

Depth values are back-projected with the camera intrinsic matrix. An extrinsic transformation then maps points into a common robot-referenced workspace. Using column-vector notation, a consistent convention is

$$ {}^{B}\mathbf{p} = {}^{B}\mathbf{T}_{C}\,{}^{C}\mathbf{p}, \qquad {}^{B}\mathbf{T}_{C} = {}^{B}\mathbf{T}_{W}\,{}^{W}\mathbf{T}_{C}, \tag{1} $$

where $C$, $W$, and $B$ denote camera, calibration-target, and robot-base frames. The transform direction and translation units must be consistent between point-cloud conversion, workspace cropping, and robot commands.

**Table 1:** Reprojection errors recorded for three intrinsic parameter sets. The original record does not fully specify the image set or aggregation protocol.

| Parameter set | Recorded reprojection error (pixels) |
|---|---|
| Calibrated | 0.1699 |
| Manufacturer-provided | 0.1943 |
| Field-of-view-derived | 0.5294 |

The calibrated intrinsics had the lowest recorded reprojection error (Table 1). Reprojection error measures calibration fit and does not establish downstream grasping performance.

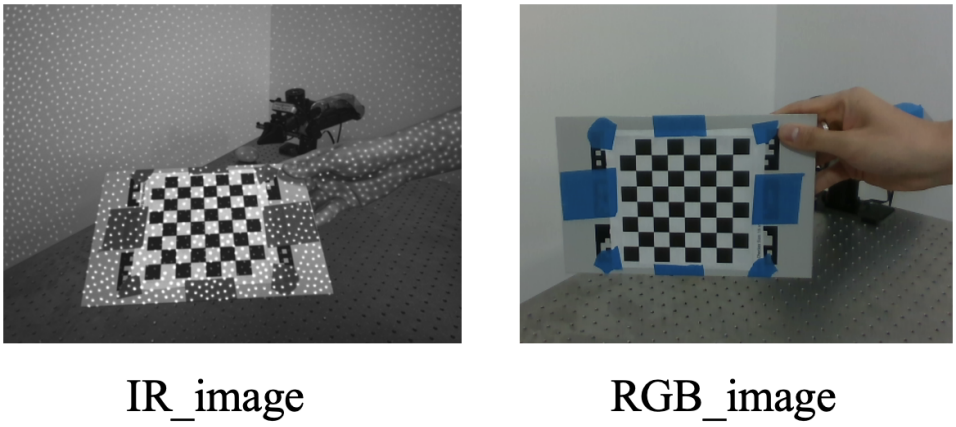
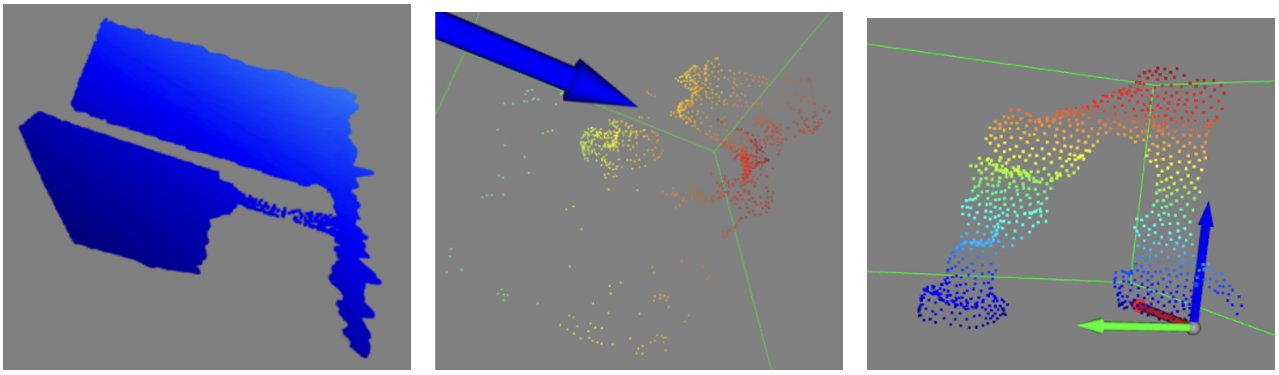



**Figure 2:** Calibration visualization retained from the original project. The diagnostic illustrates the calibration procedure; the numerical comparison is reported separately in Table 1.

### 2.3 Point-cloud preprocessing

ROS depth and joint-state streams are synchronized using message filters. Depth is converted into a point cloud with the camera calibration. The early approach attempted RANSAC plane segmentation to remove the desktop, but its recorded runtime was unsuitable for the intended online loop and its filtering could remove useful geometry.

The revised implementation first transforms points into the workspace frame, applies axis-aligned bounds, and samples 1,024 points with farthest-point sampling. The key engineering dependency is frame consistency: a workspace crop is meaningful only when its axes and bounds match the physical scene. The resulting representation is used consistently in demonstration conversion and policy execution.



**(a)** Raw depth-derived points. **(b)** Plane-segmentation attempt. **(c)** Cropped workspace.

**Figure 3:** Preprocessing diagnostics from the original project. The final policy input is sampled to 1,024 points after workspace filtering. The panels are qualitative illustrations rather than a controlled point-cloud quality benchmark.

The revised preprocessing path was recorded at 10–20 ms per frame. These exploratory timings do not include the full sensing-to-action loop, and the early RANSAC timings lack a common profiling protocol. They support a practical bottleneck diagnosis rather than a precise end-to-end speedup claim.

## 3 Policy Training and Execution

### 3.1 Learning formulation

Diffusion Policy models a conditional distribution over action sequences [1]. In DP3, a compact point-cloud representation provides spatial conditioning [2]. At time $t$, the policy is conditioned on the encoded observations $z_t$ and learns to predict noise in a perturbed demonstration action sequence:

$$\mathcal{L}(\theta) = \mathbb{E}_{t,k,\epsilon}\left[\left\|\epsilon - \epsilon_\theta(z_t, A_t^k, k)\right\|_2^2\right]. \tag{2}$$

At inference, iterative denoising produces an action sequence. Execution follows the framework's receding-horizon pattern: execute a prefix, acquire new observations, and predict again. This is an inherited policy-execution mechanism, not a new model-predictive-control contribution established by this project.

### 3.2 Recorded training settings

The original configuration describes a three-layer point encoder with widths 64, 128, and 256; 100 diffusion steps with a linear noise schedule; AdamW with learning rate $10^{-4}$; and 1,001 training epochs. These values are retained as documented configuration settings. The source does not separate training diffusion steps from the inference sampler schedule, so it should not be read as establishing 100 denoising iterations for every deployed action.

The recorded action horizon is 16 and the training configuration uses eight action steps. Related deployment notes also describe testing execution prefixes of eight, ten, and twelve steps. The available records do not isolate which prefix was used in every reported trial, and they do not provide a measured jerk comparison. Consequently, horizon selection is presented as an implementation setting rather than a proven robustness improvement.

### 3.3 Robot interface

Checkpoints are loaded for inference on a host computer, and actions are passed to the ROS arm-control interface. The original report gives inference latency below 200 ms, while related deployment notes describe an approximately 10 Hz synchronization target. Those entries do not establish sustained end-to-end 10 Hz control: perception, model execution, and communication must be timed together. Persistent stop–start policy behavior remained an observed limitation during deployment.

## 4 Task Evaluation

### 4.1 Demonstration sets

Four tasks vary the number of required actions: picking a bowl, closing a drawer, placing a lemon in a bowl, and a lemon–drawer sequence involving grasping, placement, and closure. The task-specific demonstration datasets are summarized in Table 2. Recorded episode lengths describe the dataset and should not be confused with the number of physical evaluation trials.

**Table 2:** Demonstration-set sizes transcribed from the original project record.

| Task | Demonstration episodes | Recorded steps per episode |
|---|---|---|
| Bowl picking | 20 | 150 |
| Drawer closing | 20 | 100 |
| Lemon into bowl | 40 | 300 |
| Lemon–drawer sequence | 40 | 400 |

### 4.2 Physical trials and results

Each task was evaluated in 20 physical trials. The original protocol varies bowl position and orientation, drawer opening, or lemon position as appropriate to the task. It does not specify the sampling ranges, trial reset procedure, or a precise success criterion for each task. The figures below are therefore task-specific recorded outcomes rather than a standardized benchmark or a controlled generalization comparison.

**Table 3:** Recorded real-robot task results. Completion time is averaged over successful attempts only. Counts are derived from the recorded rates and 20 trials per task.

| Task | Successful trials | Success rate | Time (s) |
|---|---|---|---|
| Bowl picking | 14/20 | 70% | 14.64 |
| Drawer closing | 20/20 | 100% | 11.14 |
| Lemon into bowl | 6/20 | 30% | 55.62 |
| Lemon–drawer sequence | 2/20 | 10% | 65.71 |

The shorter tasks had higher recorded success rates than the two multi-step tasks (Table 3). The 100% drawer result describes 20 trials in this setup; it is not a guarantee of universal reliability. Completion times cannot be compared as unconditional task duration because failed trials are excluded. In particular, the lemon–drawer average is based on only two successful attempts.

The longer lemon–drawer sequence occasionally completed, but its 2/20 outcome indicates a substantial reliability gap. The low multi-step success and persistent motion jitter remain central limitations.

## 5 Discussion and Limitations

### 5.1 What the results support

The implementation demonstrates a complete path from teleoperation data to learned actions on a physical arm. The main practical outcome is a working integration of calibration, workspace-aligned point clouds,

training data, and policy execution. Task-level evaluation shows where this integration was adequate for short behaviors and where performance deteriorated as the action sequence grew longer.

The source includes qualitative observations of repeated grasp attempts and responses to object displacement. These examples can motivate future testing but are not a controlled recovery or perturbation study. No baseline comparison establishes superiority over conventional planning, and no ablation isolates the effect of calibration, sampling, or horizon length on task success.

### 5.2 Unresolved failure modes

Possible limitations include noisy or incomplete depth observations, demonstration coverage, action-interface mismatch, and error accumulation during sequential execution. These are plausible explanations rather than experimentally isolated causes. Camera-model references alone do not demonstrate that sensor precision caused the observed failures.

Persistent policy jitter is particularly important: some successful trials still showed stop–start motion, while other attempts stalled before grasping. Success-rate reporting should therefore be paired with execution-quality measures. Future evaluation could record latency distributions, action discontinuities, object-relative errors, and the stage at which a multi-step trial fails.

### 5.3 Reproducibility priorities

A reproducible revision should pin the code and checkpoint versions; document the camera model, calibration convention, action frame, and inference sampler; define task success and reset rules; and preserve trial-level outcomes. Timing tests should state hardware, input size, and measured pipeline stages. These additions would resolve several ambiguities without presenting unperformed experiments as completed work.

## 6 Conclusion

This project implements an existing 3D Diffusion Policy framework on a physical arm and documents the integration choices required to make the data and execution paths agree. The recorded evaluation achieved 70%, 100%, 30%, and 10% success across four tasks with 20 trials each. The gap between short and sequential tasks, together with persistent jitter, makes closed-loop execution quality and evaluation design the clearest directions for further work.

## Instructor, Acknowledgments, and Code

Max Q.-H. Meng served as the course instructor. We thank Junwei Liu for guidance, and Yue Hong, Lingxiao Meng, Hao Chen, Menglian Li, and Tongxue Yue for technical assistance and suggestions, as acknowledged in the original project report.

The original project identifies the ARX DP3 training code and ARX remote-control code. A project materials repository provides the report and presentation. These resources and the original DP3 method should be credited separately from the implementation work described here.

## References

[1] C. Chi et al. “Diffusion Policy: Visuomotor Policy Learning via Action Diffusion.” *Robotics: Science and Systems*, 2023.

[2] Y. Ze et al. “3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations.” arXiv:2403.03954, 2024. https://arxiv.org/abs/2403.03954.